

AI协作反思日志 #4

AI 协作反思日志 — Phase 4 核心开发与功能完善

团队：Action 团队 日期：2026-06-07

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
Sprint 合并规划	ChatGPT、Claude	将 P0/P1/P2 功能拆成 2-4 小时任务，识别跨模块依赖、延期项和集成风险	4	20%
仓库结构与接口核验	Codex、Claude Code	检查真实目录、路由、API 封装、Controller、DTO、Service 和 CI 配置，避免按旧文档误判	4	20%
用户身份与登录态检查	Codex、Claude Code	核验 Token 传递、角色切换、刷新后的登录态和越权访问	4	25%
需求大厅与响应流程	Codex、ChatGPT	检查发布需求、列表筛选、详情、服务方响应和字段映射	4	30%
会话、报价与订单状态流转	Codex、Claude、ChatGPT	审查订单状态机、合法状态转换、角色权限、重复操作和通知触发	4	35%
交付模块	ChatGPT、Codex	生成并审查 Delivery 实体、DTO、Repository、Service、Controller 和订单状态推进对接	4	30%
评价、信用与通知	ChatGPT、Claude Code、Codex	检查双向评价、重复评价拦截、信用记录、通知触发和列表查询	4	35%
动态广场	ChatGPT、Codex、Claude Code	基于现有 social 骨架审查内存实现、持久化改造、页面对接和修改边界	4	35%
单元测试与集成测试	ChatGPT、Claude、Codex	生成并审查正常路径、权限异常、重复操作、非法状态和前后端字段对齐测试	4	30%
CI/CD 检查	Codex、Claude Code	检查 Maven 测试、构建、MySQL 服务、流水线文件和前端构建命令	4	20%
Bug 日志与演示说明	ChatGPT、Claude	按问题现象、根因、修复方案、验证结果整理记录，补充演示路径	5	15%

2. AI 助力分析 (Q1)

AI 在编码中最有价值的场景：AI 能快速生成模块骨架、DTO、Service、Controller、测试样例和接口清单，并能根据现有仓库文件定位需要修改的位置。以交付模块和动态模块为例，AI 能较快识别跨模块依赖、已有骨架和缺失能力，减少重复性编码工作。但团队仍需人工确认权限、事务、状态机和修改边界。

AI 在测试生成中最有价值的场景：AI 对边界条件和异常路径的补充较有效，例如未登录访问、越权操作、重复评价、重复点赞、非法状态流转、刷新后状态丢失、前后端字段不一致等。人工重点检查测试是否真正断言关键结果，避免出现只有接口调用、没有有效验证的空洞测试。

AI 在调试中最有价值的场景：AI 对编译错误、路由引用错误、事件绑定失效、字段命名不一致、状态未刷新、旧页面仍被引用、CI 命令与目录不匹配等问题定位较快。对于跨模块状态机、业务范围取舍和是否允许修改公共文件，仍需人工判断。

3. AI 误导分析 (Q2)

"AI 代码信任度实验"数据摘要：

- AI 直出的测试通过率：约 50%
- 人工审查修复后的测试通过率：约 90%
- 发现的主要问题类型：权限校验不完整、状态迁移过于宽松、重复操作拦截不足、事务边界不清晰、下游通知和交付评价衔接遗漏、正常路径测试较多但异常路径不足。

"AI 调试对决"实验总结：

- AI 擅长调试的 Bug 类型：编译错误、路由或组件引用错误、前后端字段不一致、动态节点事件绑定失效、重复提交、静态数组或 localStorage 伪状态残留、CI 命令路径错误。
- AI 不擅长调试的 Bug 类型：跨模块状态机、产品范围判断、接口归属边界、是否保留兼容逻辑、是否允许修改共享模块。
- AI 修复方案的典型问题：只修表象、不检查根因；忽略现有目录和分工边界；一次修改过多文件；为快速通过测试而弱化业务校验；将原型逻辑误当成正式实现。

4. Prompt 改进计划 (Q3)

上阶段改进措施的验证结果：将“直接生成代码”改为“先阅读仓库、输出审查结论、列出修改文件、确认边界后再编码”后，AI 误改公共模块和重复实现的情况明显减少。要求 AI 在提交前执行 `git status`、`git diff --stat` 和构建测试，也提高了修改可追溯性。

编码阶段的 Prompt 最佳实践：明确当前分支、允许修改目录、禁止修改目录、可复用模块、接口契约、数据库变化、验收标准和测试命令；要求 AI 先列出文件清单和影响范围，得到确认后修改；每次只完成一个小任务。

调试阶段的 Prompt 最佳实践：同时提供报错信息、复现步骤、相关接口、当前分支、最近 diff、涉及文件和预期结果；要求 AI 区分环境问题、前端问题、后端问题、数据库问题和跨模块依赖问题；要求给出根因、最小修复方案和验证方式。

下阶段计划改进：继续使用 ChatGPT 和 Claude 做需求复核，使用 Codex 和 Claude Code 在仓库内执行和交叉检查；补齐前端构建检查、集成测试记录、CI 截图和演示账号；合并前
坚持人工检查 Git diff、运行测试并走查核心演示路径。

5. 本阶段的核心工程判断

决策内容：以真实仓库代码为准，不沿用已经过时的暂定技术栈描述；优先保证注册登录、需求发布、响应、报价、订单、交付、评价、信用和通知主链路可运行；动态功能优先完成基础闭环和持久化，举报、复杂审核、评论、回复、转发、视频等功能延期；原型页面不能直接作为正式交付，正式前端必须调用真实接口并保证刷新后状态一致。

为什么 AI 无法单独做这个决策：AI 可以快速生成方案，但无法仅凭局部代码准确判断课程交付范围、团队分工边界、跨模块风险和演示优先级。不同 AI 还可能给出相互冲突的建议，例如扩大功能范围、重构已有结构或直接删除旧兼容逻辑。

团队最终如何判断和取舍：团队结合 P4 要求、任务看板、仓库真实结构、接口依赖和时间限制进行人工确认。所有新增功能先做最小可用版本；共享模块修改必须说明影响范围；合并前检查 Git diff、运行构建和测试、核对数据库状态，并按核心演示路径进行人工验证。